The easiest way to create a custom Tag Helper is to derive from the `TagHelper` base class and override the `Process()` or `ProcessAsync()` function that describes how the class should render itself. The following listing shows your complete custom Tag Helper, `SystemInfoTagHelper`, which renders the system information to a `<div>`. You could easily extend this class if you wanted to display additional fields or add options.

Listing 32.1 `SystemInfoTagHelper` to render system information to a view

```
public class SystemInfoTagHelper : TagHelper                          ❶
{
    private readonly HtmlEncoder _htmlEncoder;                         ❷
    public SystemInfoTagHelper(HtmlEncoder htmlEncoder)                ❷
    {
        _htmlEncoder = htmlEncoder;
    }

    [HtmlAttributeName("add-machine")]                                 ❸
    public bool IncludeMachine { get; set; } = true;

    [HtmlAttributeName("add-os")]                                      ❸
    public bool IncludeOS { get; set; } = true;

    public override void Process(                                      ❹
        TagHelperContext context, TagHelperOutput output)              ❹
    {
        output.TagName = "div";                                        ❺
        output.TagMode = TagMode.StartTagAndEndTag;                    ❻
        var sb = new StringBuilder();

        if (IncludeMachine)                                            ❼
        {                                                              ❼
           sb.Append(" <strong>Machine</strong> ");                    ❼
           sb.Append(_htmlEncoder.Encode(Environment.MachineName));    ❼
        }                                                              ❼

        if (IncludeOS)                                                 ❽
        {                                                              ❽
```